SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA04
COURSE NAME: Operating Systems

COURSE OUTCOME COVERED

CO1: *Apply the concepts of Operating System types, structures, and system calls to demonstrate their functions in various computing environments.*

QUESTIONS: 5 **Total Marks:100**

Annexure A

SHORT ANSWER TEST

Code of Conduct:

I Ramya.M (192126014 8) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

45/50 90/100

Signature of the student:

Ramya.M

Annexure A

Short Answer Test - Question Paper

Question 1 (5 Marks)

Suppose a modern Operating System supports multiple users and processes with shared resources like CPU, memory, and I/O devices.

(a) Classify different types of Operating Systems suitable for this scenario and justify your choice.

(b) Draw the structure of an Operating System showing how resources are managed.

(c) Write a sequence of system calls required to create and execute a new process in such an environment. (BL2)

Question 2 (5 Marks)

Consider an Operating System managing multiple files and devices for concurrent users.

(a) Categorize system calls used for file and device management.

(b) Illustrate how the OS handles file operations using a layered structure diagram.

(c) Write a pseudo-code or C-based system call sequence to open, read, and close a file. (BL3)

Question 3 (5 Marks)

Assume a real-time Operating System used in an embedded system controlling industrial machinery.

(a) Identify the type of OS and explain why it is suitable for real-time applications.

(b) Draw a diagram representing the interaction between hardware, kernel, and user processes.

(c) List and explain system calls required for process synchronization in this system. (BL2)

Question 4 (5 Marks)

A distributed computing system consists of multiple interconnected computers sharing resources over a network.

(a) Classify the type of Operating System used and compare it with centralized OS.

(b) Draw the architecture of a distributed OS showing communication between nodes.

(c) Write the sequence of system calls involved in inter-process communication (IPC). (BL3)

Question 5 (5 Marks)

Consider a scenario where an Operating System must manage process scheduling, memory allocation, and device communication efficiently.

(a) Explain the major functions of an OS in handling these tasks.

(b) Draw a block diagram representing OS components and their interactions.

(c) Write system call examples for memory allocation and device management. (BL3)

Rubrics for Calculation

Criteria	Weightage (Marks)	Level 4 - Exemplary (Full Marks)	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Conceptual Understanding	35	Demonstrates clear, complete, and in-depth understanding of OS concepts (types, structure, system calls)	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answers	25	All answers are correct, precise, and relevant	Mostly correct with minor errors	Partially correct with noticeable errors	Mostly incorrect or irrelevant
Application of Knowledge	15	Effectively applies concepts to scenarios/questions	Applies with minor mistakes	Limited or partial application	No meaningful application
Completeness of Response	15	All parts of each question are fully answered	Most parts covered with small omissions	Some parts missing	Incomplete answers
Clarity & Presentation	10	Clear, well-structured, and neatly presented answers	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Annexure A

Answers - CSA04 Operating Systems (CO1)

Question 1 (5 Marks) - BL2

Scenario: A modern OS supports multiple users and processes with shared resources like CPU, memory and I/O devices.

(a) Types of Operating Systems suitable for this scenario

Type	Key idea	Suitable here?
Multiprogramming OS	Several jobs kept in memory; CPU switches when a job waits for I/O	Yes - keeps CPU busy
Time-sharing (multitasking) OS	CPU time sliced among users/processes, giving quick response	Yes - best fit
Multi-user OS	Many users logged in at once with separate accounts and protection	Yes - required
Batch OS	Jobs run one after another with no interaction	No - no interactivity
Real-time OS	Guaranteed response deadlines	No - not needed

Choice and justification: A multi-user, time-sharing (multiprogramming) OS such as Linux/UNIX or Windows Server is the best choice. Memory holds many processes, the scheduler gives each a short time quantum so every user feels the system is theirs alone, and the OS protects each user's files and memory from others. Shared CPU, memory and I/O devices are allocated fairly and utilisation stays high.

(b) Structure of the Operating System showing resource management

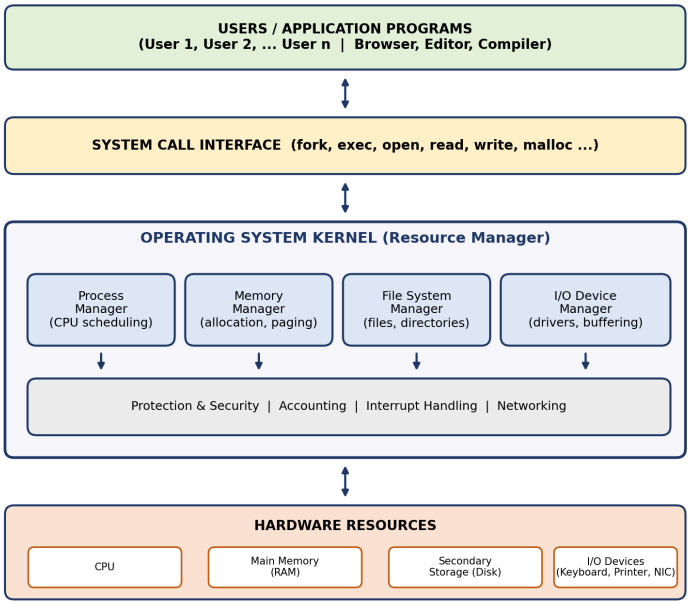


Figure 1: Layered structure of an OS and its resource managers

Users and applications request services only through the system call interface. The kernel contains one manager per resource: the Process Manager shares the CPU, the Memory Manager shares RAM, the File

System Manager shares storage, and the I/O Manager shares devices through drivers. Protection and accounting services run across all of them.

(c) System call sequence to create and execute a new process

Step	System call	Purpose
1	fork()	Parent creates a child process (copy of itself); returns 0 to child and child PID to parent
2	exec() family (execlp)	Child replaces its memory image with a new program
3	wait()	Parent blocks until the child terminates
4	exit()	Child ends, releases resources, returns status to parent

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();              // 1. create process
    if (pid < 0) { perror("fork failed"); return 1; }
    else if (pid == 0) {             // child
        execlp("/bin/ls", "ls", "-l", NULL);   // 2. load new program
        perror("exec failed");
    } else {                         // parent
        wait(NULL);                  // 3. wait for child
        printf("Child complete\n");
    }
    return 0;                        // 4. exit()
}
```

Question 2 (5 Marks) - BL3

Scenario: An OS manages multiple files and devices for concurrent users.

(a) Categories of system calls for file and device management

Operation	File management	Device management
Create / acquire	create(), open()	request() / open() the device
Release	close(), delete()	release() / close() the device
Data transfer	read(), write()	read(), write()
Positioning	lseek() (reposition)	reposition (where applicable)
Attributes	get / set file attributes (stat, chmod)	get / set device attributes (ioctl)
Logical attach	mount(), unmount()	attach / detach devices logically

(b) File operations using a layered structure

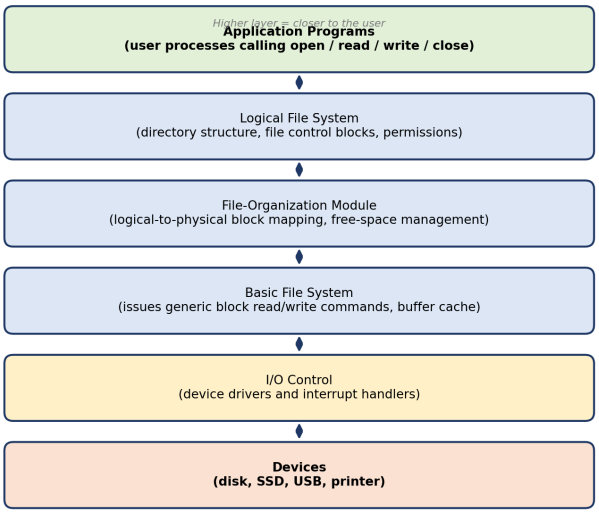


Figure 2: Layered file-system structure

A call such as read() travels downward: the logical file system finds the file control block and checks permissions, the file-organization module converts the logical block to a physical block, the basic file system issues a generic block command, and I/O control (driver) talks to the disk. Data returns upward through the same layers. Layering simplifies design and lets the same upper layers work with different devices.

(c) System call sequence to open, read and close a file

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int main() {
    char buf[100];
    int fd = open("data.txt", O_RDONLY);     // open
    if (fd < 0) { perror("open"); return 1; }

    ssize_t n = read(fd, buf, sizeof(buf)-1); // read
    if (n > 0) { buf[n] = '\0'; printf("%s", buf); }

    close(fd);                                // close
    return 0;
}
```

Sequence: open() returns a file descriptor, read() copies bytes from the file into the buffer using that descriptor, and close() frees the descriptor and kernel resources.

Question 3 (5 Marks) - BL2

Scenario: A real-time OS in an embedded system controlling industrial machinery.

(a) Type of OS and why it suits real-time applications

Type: Hard Real-Time Operating System (RTOS), for example VxWorks, QNX, FreeRTOS or RTLinux.

Why it is suitable:

- **Deterministic response:** tasks are guaranteed to finish before their deadlines; a late result is a system failure (e.g., a press must stop within milliseconds).
- **Priority-based preemptive scheduling:** a high-priority task (emergency stop) immediately pre-empts a low-priority one.
- **Low interrupt latency and minimal jitter:** sensor events are handled in predictable, bounded time.
- **Small footprint and reliability:** runs on limited embedded hardware and works continuously without crashing.

(b) Interaction between hardware, kernel and user processes

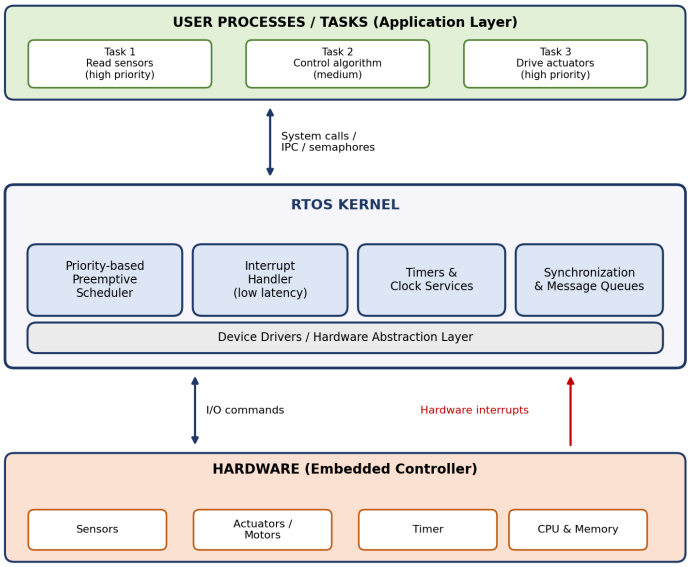


Figure 3: Hardware - RTOS kernel - user task interaction

User tasks request kernel services through system calls. The kernel schedules tasks by priority and drives the hardware through drivers. Sensors and timers raise hardware interrupts that go straight to the kernel's interrupt handler, which wakes the proper task.

(c) System calls for process synchronization

System call	Explanation
sem_init() / sem_open()	Creates and initialises a semaphore with a starting count (1 for mutual exclusion).
sem_wait() (P / wait)	Decrements the semaphore; blocks the task if the count is 0 (resource busy).
sem_post() (V / signal)	Increments the semaphore and wakes one waiting task.

System call	Explanation
pthread_mutex_lock() / unlock()	Locks and unlocks a mutex so only one task is in the critical section; many RTOSs add priority inheritance.
mq_send() / mq_receive()	Message queue calls to pass data and synchronise producer and consumer tasks.
sem_destroy()	Deletes the semaphore when it is no longer needed.

```
sem_wait(&mutex);      // enter critical section (e.g., shared sensor buffer)
   /* update shared data */
sem_post(&mutex);      // leave critical section
```

Question 4 (5 Marks) - BL3

Scenario: A distributed system of interconnected computers sharing resources over a network.

(a) Type of OS and comparison with a centralized OS

Type: Distributed Operating System. A loosely coupled set of independent computers appears to users as a single coherent system (single-system image), with resources shared transparently. Examples: Amoeba, LOCUS, Plan 9, and cluster/cloud operating platforms.

Feature	Centralized OS	Distributed OS
Structure	One computer with one kernel	Many computers, each with a kernel plus distributed layer
Resources	Local to a single machine	Pooled and shared across nodes
Communication	Shared memory / local IPC	Message passing over network
Fault tolerance	Single point of failure	A node can fail; others continue
Scalability	Limited by one machine	Scales by adding nodes
Performance	Bounded by one CPU	Parallel processing and load balancing
Complexity	Simple to design and manage	Complex (synchronisation, security, latency)

(b) Architecture of a distributed OS

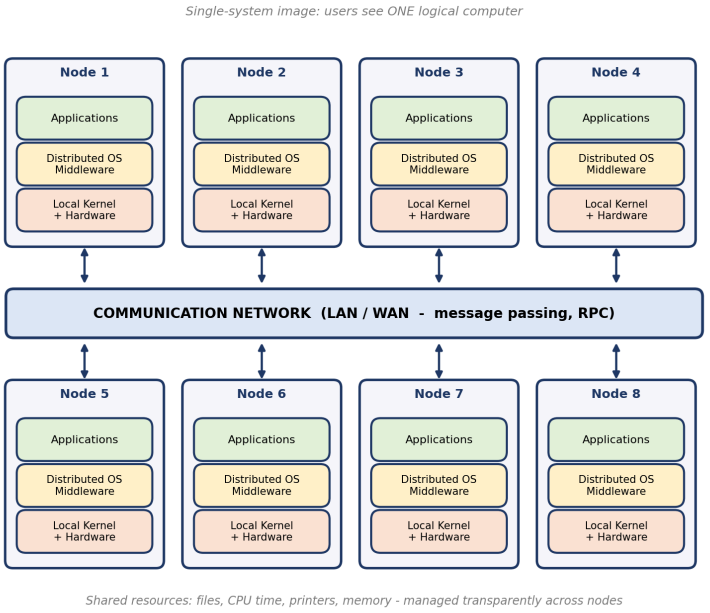


Figure 4: Distributed OS architecture with inter-node communication

Each node runs a local kernel and a distributed OS layer (middleware). Nodes cooperate only by exchanging messages across the network, using RPC or sockets, so processes on different machines can share files, devices and load.

(c) System call sequence for inter-process communication (IPC)

Message passing between processes on different nodes (client-server sockets):

Step	Server	Client
1	socket() - create endpoint	socket() - create endpoint
2	bind() - attach address/port	-
3	listen() - wait for connections	-
4	accept() - accept a client	connect() - connect to server
5	recv() - receive request	send() - send request
6	send() - send reply	recv() - receive reply
7	close()	close()

Local IPC alternatives: pipe() / read() / write() for parent-child pipes, and shared memory using shmget() -> shmat() -> (use memory) -> shmdt() -> shmctl().

Question 5 (5 Marks) - BL3

Scenario: The OS must manage process scheduling, memory allocation and device communication efficiently.

(a) Major functions of the OS

- **Process scheduling:** maintains PCBs and ready/wait queues, and the CPU scheduler selects the next process (FCFS, SJF, Round Robin, Priority). The dispatcher performs the context switch. Process creation, termination and synchronization are also handled here.
- **Memory allocation:** tracks which memory parts are used and by whom, allocates and de-allocates space (contiguous allocation, paging, segmentation), provides virtual memory, and protects each process's address space.
- **Device communication:** hides hardware details through device drivers, uses interrupts and DMA, provides buffering, caching and spooling, and gives a uniform interface (open, read, write) for all devices.
- **Supporting functions:** file management, protection and security, error handling and resource accounting.

(b) Block diagram of OS components and interactions

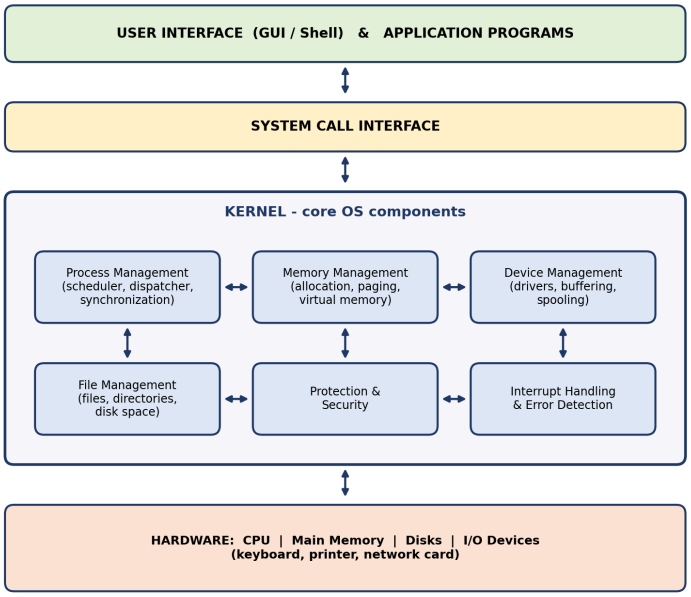


Figure 5: OS components and their interactions

The process manager asks the memory manager for space when it creates a process; the file manager and device manager cooperate through drivers to move data; the protection module checks every request; and interrupts from hardware flow to the kernel for handling.

(c) System call examples for memory allocation and device management

System call	Use
malloc() / calloc()	Library routines that request heap memory (internally call brk() / sbrk() or mmap()).
brk() / sbrk()	Change the size of the process data segment (heap).

System call	Use
mmap() / munmap()	Map or unmap memory regions or files into the address space.
free()	Return allocated memory to the system.
open() / close()	Acquire and release a device file (e.g., /dev/ttyS0).
read() / write()	Transfer data from or to the device.
ioctl()	Get or set device-specific parameters (e.g., baud rate).

```
/* Memory allocation */
int *arr = (int *) malloc(10 * sizeof(int));   // allocate
if (arr == NULL) { perror("malloc"); return 1; }
arr[0] = 5;
free(arr);                                      // release

/* Device management */
int fd = open("/dev/ttyS0", O_RDWR);   // request device
ioctl(fd, TCGETS, &settings);         // get/set attributes
write(fd, "Hello", 5);                // write to device
read(fd, buf, sizeof(buf));           // read from device
close(fd);                            // release device
```